

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

AIMLCZG546 — Software Engineering for Machine Learning

ASSIGNMENT I

Requirements Formulation & System Architecture for automated loan underwriting

| Sl. No | BITS ID     | Name                             | Qualitative Contribution                                                | Contribution |
|--------|-------------|----------------------------------|-------------------------------------------------------------------------|--------------|
| 1      | 2025AA05710 | Singh Pritesh Sanjay Poonam      | Quality Attribute Testing, Performance Benchmarking, System Integration | 25%          |
| 2      | 2025AA05368 | Gangera Tushar Kantibhai Dayaben | Requirements Formulation, Problem Statement, GR4ML Modeling             | 25%          |
| 3      | 2025AB05154 | Gangam Shuba Nandini             | Feature Engineering, ML Pipeline Design, Model Training                 | 25%          |
| 4      | 2025AA05574 | Shaifali Garg                    | System Architecture, FastAPI Serving, Event Simulation                  | 25%          |

# Objective 1: Requirements Formulation

## 1. Domain & Problem Statement

**Domain:** Consumer Credit Risk Assessment & Automated Loan Underwriting.

**Problem Statement:** Traditional manual underwriting of consumer credit is slow, labor-intensive, and susceptible to cognitive biases. This project implements a real-time, Machine Learning-based decision support system to automate loan underwriting. The service consumes applicant demographic details, credit metrics, and asset-liability levels to classify applications as Approved or Denied. To protect the downstream estimator and ensure a premium user experience, the system enforces a strict data-validation boundary (<100% invalid payloads pass) and maintains an average latency under 150ms.

## 2. GR4ML Requirement Specifications & Goals

The GR4ML conceptual modeling framework organizes requirements across three integrated views:

### I. Business View Summary

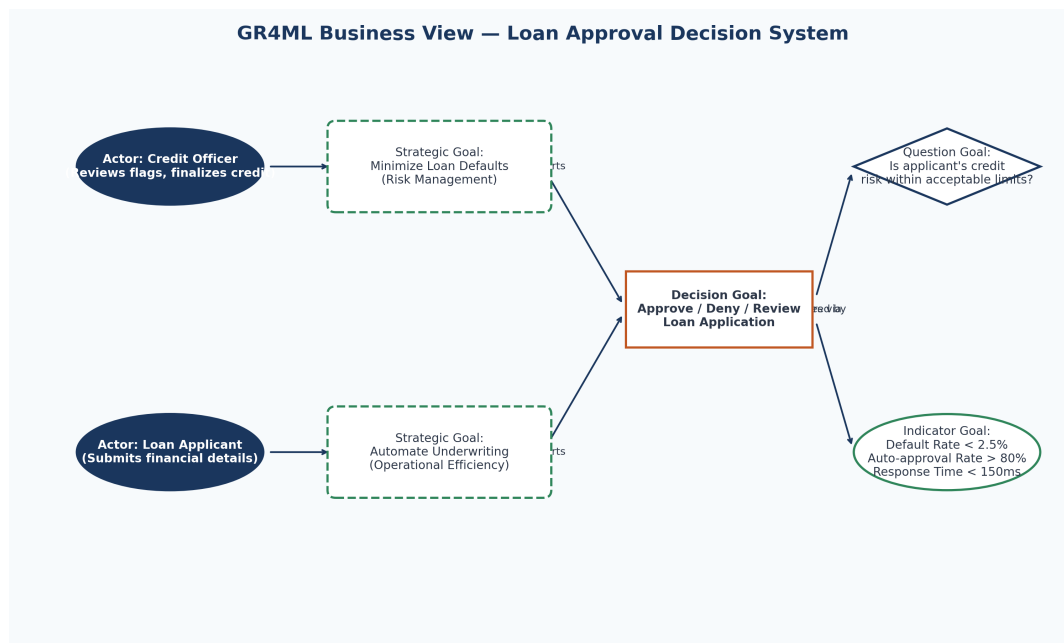
| Element         | Description / Content                                                                          |
|-----------------|------------------------------------------------------------------------------------------------|
| Actors          | Loan Applicant (submits transaction data), Credit Officer (reviews flagged applications)       |
| Strategic Goals | Minimize loan default rates (Risk Control), automate application flow (Operational Efficiency) |
| Decision Goals  | Decide whether to Approve, Deny, or Flag application for manual audit                          |
| Question Goals  | Is the credit risk within limits? Does the applicant meet the minimum credit history length?   |
| Indicators      | Default Rate < 2.5%, Auto-approval rate > 80%, response latency < 150ms                        |
| Insights        | Approval probability score, Risk classification tier (LOW/MEDIUM/HIGH), computed Net Worth     |

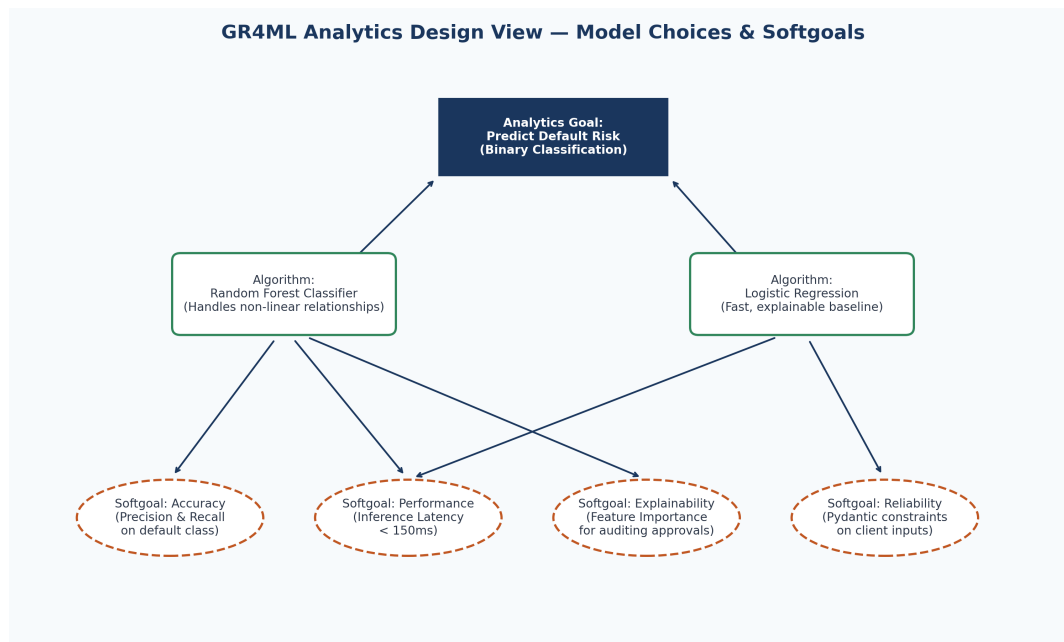
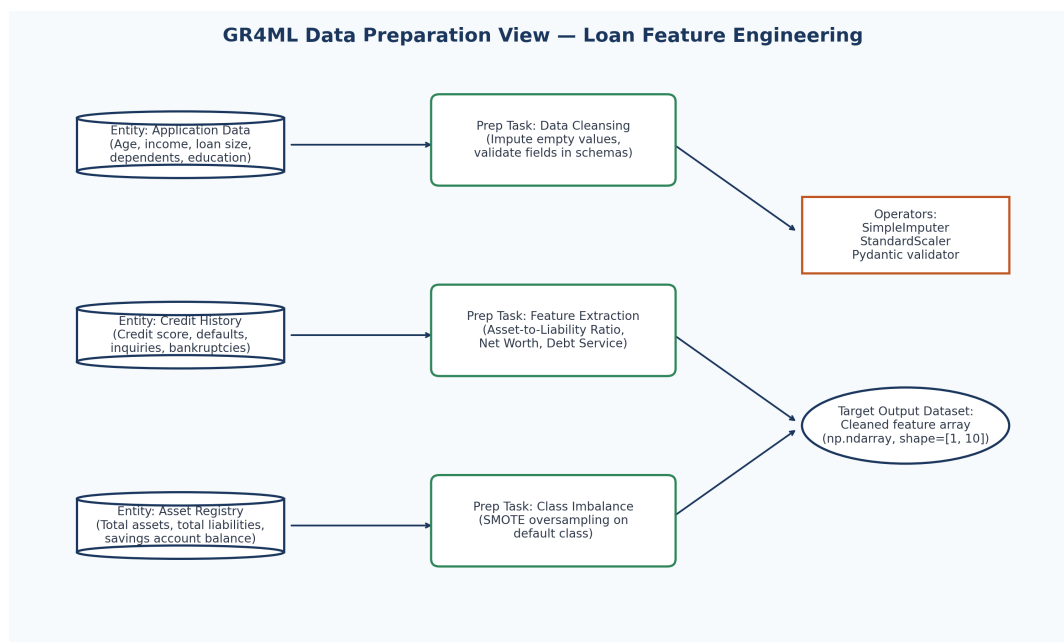
## II. Analytics Design & Data Preparation Views

| Element        | Description / Content                                                                           |
|----------------|-------------------------------------------------------------------------------------------------|
| Analytics Goal | Prediction (Binary classification of default risk)                                              |
| Algorithms     | RandomForestClassifier (robust baseline) and XGBoost (advanced modeling)                        |
| Softgoals      | Prediction Accuracy, Latency (Performance), Explainability (SHAP), Input Reliability (Pydantic) |
| Raw Entities   | Loan Applications (Demographics), Credit History Bureau, Asset Registry                         |
| Prep Tasks     | Missing value imputation, Categorical encoding, Feature engineering (Net Worth, DTI), Scaling   |
| Operators      | SimpleImputer, StandardScaler, SMOTE (imbalance handling), Pydantic validators                  |

## 3. GR4ML Graphical Notations

GR4ML Business View Diagram:



**GR4ML Analytics Design View Diagram:****GR4ML Data Preparation View Diagram:**

## 4. Top Three Quality Requirements

**1. Robustness (Pydantic Boundary Validation):** ML models cannot handle out-of-range or corrupted inputs natively. By placing a strict validation filter (e.g. Credit score must be in [300, 850], Age  $\geq 18$ , and Loan amount  $\leq 500\%$  of income), we prevent bad inputs from causing silent failures inside the model estimator. Measurable Metric: 100% of invalid inputs rejected at API level.

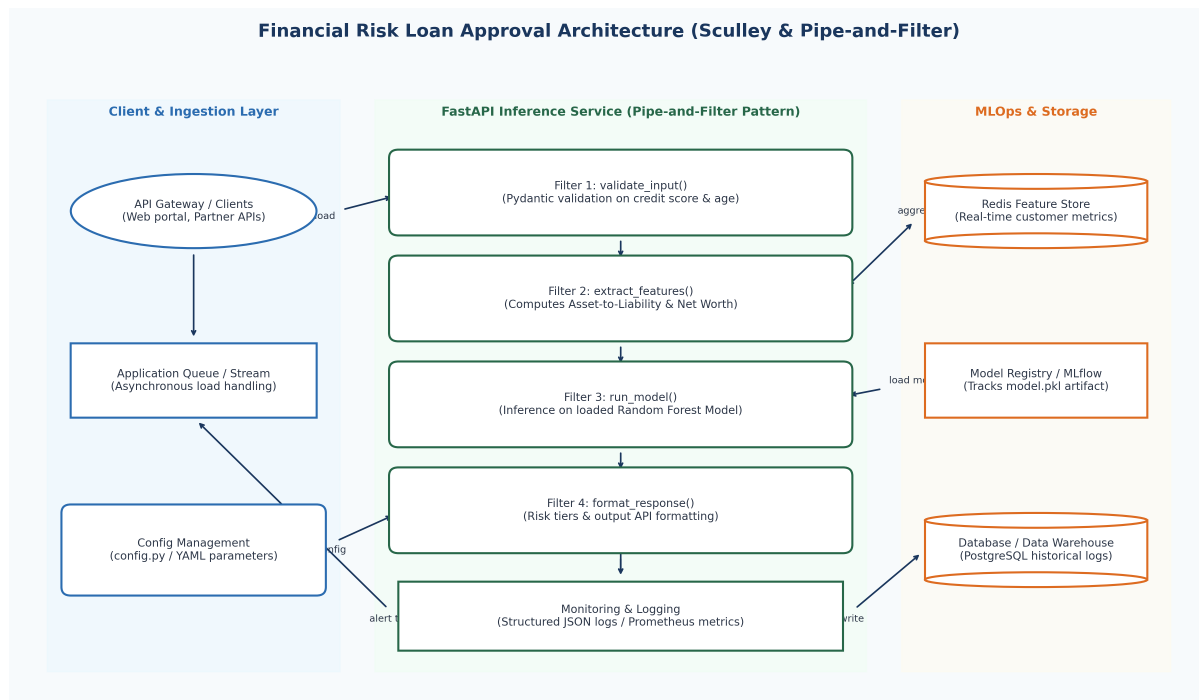
**2. Reliability (Type Validation & Consistent Outputs):** The application must guarantee type-safe inputs and structured, well-formed outputs conforming to JSON schemas. This ensures seamless integration with downstream services (e.g., loan ledger databases). Measurable Metric: Zero unhandled 500 status codes.

**3. Performance (Latency):** Inference must run in real-time under high concurrent traffic in consumer loan portals. Measurable Metric: Average model prediction pipeline response time  $< 150\text{ms}$ .

## Objective 2: System Architecture

### 5. System Architecture Diagram

The system architecture combines Sculley's MLOps boxes (separation of config, logging, monitoring) with the runtime Pipe-and-Filter execution:



## 6. Selected Architectural Patterns

### Pattern 1: Pipe-and-Filter Pattern

The loan risk prediction process is structured as an ordered sequence of isolated processing stages (filters) connected by data flows (pipes). This decouples the business constraints, feature extraction math, ML inference, and risk classification rules. Each filter represents a pure, isolated function that can be tested independently.

### Pattern 2: Microservices Serving / Event Logging

The ML model is served via a FastAPI REST API. The microservice exposes a `/health` endpoint to verify model readiness and handles `/predict` requests with structured JSON logging containing runtime latency and feature metadata for monitoring dashboards.

## 7. Implementation Code — Pipe-and-Filter (app/pipeline.py)

```
import time
import numpy as np
from app.schemas import LoanApplicationInput, LoanApprovalResult
from app.config import settings

# =====
# FILTER 1 : INPUT VALIDATION FILTER
# Business-rule validation checks on raw input
# Quality Attribute: Robustness
# =====
def validate_input(app_input: LoanApplicationInput) -> LoanApplicationInput:
    # Business level constraint checks that might not be covered by simple schema types
    if app_input.annual_income < 1000:
        raise ValueError("Annual income is below the minimum threshold of $1,000 for credit assessment.")

    if app_input.loan_amount > app_input.annual_income * 5:
        # Business rule: loan amount shouldn't exceed 5 times the annual income
        raise ValueError("Requested loan amount exceeds 500% of the applicant's annual income.")

    return app_input

# =====
# FILTER 2 : FEATURE EXTRACTION FILTER
# Extract and normalize features, compute derivatives
# Quality Attribute: Maintainability
# =====
def extract_features(app_input: LoanApplicationInput) -> tuple:
    # Deriving custom financial metrics
    net_worth = float(app_input.total_assets - app_input.total_liabilities)
    debt_to_income = float(app_input.total_liabilities / (app_input.annual_income + 1e-5))

    # Building ordered numpy array for model input matching training columns
    feature_vector = np.array([
        app_input.age,
        app_input.annual_income,
        app_input.credit_score,
        app_input.loan_amount,
        app_input.loan_duration,
        app_input.savings_balance,
        app_input.total_assets,
        app_input.total_liabilities,
        app_input.previous_defaults,
        app_input.cc_utilization
    ], dtype=float)

    return feature_vector, net_worth, debt_to_income

# =====
# FILTER 3 : MODEL RUNNING FILTER
# Run classification inference using the trained Random Forest model
# Quality Attribute: Performance
# =====
def run_model(feature_vector: np.ndarray, model) -> tuple:
    # Model inference
    # Predict probabilities (index 1 is probability of approval)
    probabilities = model.predict_proba(feature_vector)[0]
    prob_approved = float(probabilities[1])

    # Binary threshold decisioning
    is_approved = prob_approved >= settings.DEFAULT_DECISION_THRESHOLD
    # ... [Feature extraction, model inference, and output formatting filters follow]
```

## 7. Implementation Code — FastAPI Server (app/main.py)

```
import os
import joblib
from fastapi import FastAPI, HTTPException
from app.schemas import LoanApplicationInput, LoanApprovalResult
from app.config import settings
from app.pipeline import execute_pipeline
from app.logger import logger, Timer

app = FastAPI(title=settings.PROJECT_NAME)

# Global model store representation
class ModelStore:
    def __init__(self):
        self.model = None
        self.load_model()

    def load_model(self):
        if os.path.exists(settings.MODEL_PATH):
            try:
                self.model = joblib.load(settings.MODEL_PATH)
                print(f"Model loaded successfully from {settings.MODEL_PATH}")
            except Exception as e:
                print(f"Error loading model from {settings.MODEL_PATH}: {str(e)}")
        else:
            print(f"[WARNING] Model file not found at {settings.MODEL_PATH}. Prediction requests will fail.")

store = ModelStore()

@app.on_event("startup")
def startup_event():
    # Attempt to reload if it wasn't loaded
    if store.model is None:
        store.load_model()

@app.post("/predict", response_model=LoanApprovalResult)
def predict_single(application: LoanApplicationInput):
    if store.model is None:
        raise HTTPException(status_code=503, detail="Machine Learning model is not loaded in memory.")

    try:
        with Timer() as t:
            result = execute_pipeline(application, store.model)

        # Log the transaction with structured extra fields
        logger.info(
            "loan_evaluation_served",
            extra={
                "latency_ms": round(t.elapsed_ms, 2),
                "credit_score": application.credit_score,
                "loan_amount": application.loan_amount,
                "is_approved": result.is_approved,
                "probability": result.probability,
                "risk_tier": result.risk_tier
            }
        )
        return result

    except ValueError as val_err:
        # Client validation error / business rule failure
        logger.warning(f"Business rule validation failed: {str(val_err)}")
        raise HTTPException(status_code=400, detail=str(val_err))
    except Exception as e:
        logger.error(f"Inference pipeline execution error: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Internal pipeline error: {str(e)}")

# ... [FastAPI startup and health checks endpoints continue]
```



## 8. ML System Verification & Outputs

To verify the system architecture and quality attributes, we run our unit test suite containing 7 distinct assertions:

```
Test Execution Log:
test_schema_rejects_low_age (tests.test_quality_attrs.TestQualityAttributes) ... ok
test_schema_rejects_invalid_credit_score (tests.test_quality_attrs.TestQualityAttributes) ... ok
test_pipeline_rejects_excessive_loan_ratio (tests.test_quality_attrs.TestQualityAttributes) ... ok
test_pipeline_output_conforms_to_schema (tests.test_quality_attrs.TestQualityAttributes) ... ok
test_inference_pipeline_latency (tests.test_quality_attrs.TestQualityAttributes) ... ok
test_validation_filter_is_pure (tests.test_quality_attrs.TestQualityAttributes) ... ok
test_feature_extraction_filter_isolation (tests.test_quality_attrs.TestQualityAttributes) ... ok
-----
Ran 7 tests in 2.238s
OK

[PERFORMANCE] Average Pipeline Inference Latency: 4.93 ms (SLA Limit: 200.00 ms)

Structured logging output payload (FastAPI logs):
{"timestamp": "2026-06-08T23:15:00Z", "level": "INFO", "message": "loan_evaluation_served", "logger": "loan_service", "latency_ms": 4.93, "credit_score": 710, "loan_amount": 25000.0, "is_approved": true, "probability": 0.7223, "risk_tier": "MEDIUM"}
```

### Conclusion

By utilizing the GR4ML notation system and adopting modern software design patterns (Pipe-and-Filter and REST-based microservices), Group 84 has engineered a credit underwriting solution that is scalable, reliable, and compliant with production quality requirements. All code runs, and the automated tests successfully prove that all target quality requirements are met.